PennyPal — Technical Documentation Report

Make Every Penny Count • Software Requirements & Architecture Specifications

🏆 TechWiz 7 — World Tech Championship | Theme: NextGen BudgetBee

Category: Multi-Platform App Computing | Version: 1.0.0 (Release Build 1)

📌 **Submission Compliance Notice (SRS Section 1.9):** This document is the formal technical documentation report for the PennyPal platform. It satisfies all 7 mandated components of Section 1.9 (Problem definition, design specifications, DFD/flowcharts, database design & data dictionary, test datasets, deployment instructions, and official credentials). In accordance with competition rules, this document contains **no raw source code**.

Table of Contents

1. Executive Summary & Problem Definition
2. Project Scope & Constraints
3. System Architecture, Design Specifications & Screen Showcase
4. Process Flowcharts & Data Flow Diagrams (DFDs)
5. Database Design & Data Dictionary
6. Comprehensive Test Data & Verification Suite
7. Installation & Deployment Instructions (MANDATORY)
8. User Credentials & Role Privileges (MANDATORY)
9. AI Tools & Technology Acknowledgement (SRS Page 11 Compliance)
10. Assumptions, Security & Scope Boundaries

---

1. Executive Summary & Problem Definition

1.1 Background & Necessity (SRS Section 1.1)

Financial literacy is a critical yet routinely neglected life skill among tertiary, undergraduate, and vocational students. Upon entering college or university, students receive disposable funds from allowances, scholarships, student loans, stipends, internships, or part-time employment. However, the vast majority possess little to no formal training in personal budgeting, cash flow tracking, or disciplined savings. Daily micro-transactions—such as cafeteria lunches, transportation, university photocopying, study materials, digital subscriptions, campus events, and social outings—frequently deplete student funds early in the academic month. Traditional tracking methods (such as physical notebooks, paper diaries, or generic desktop spreadsheets) suffer from severe limitations:

- **High Friction:** Manual entry in spreadsheets is cumbersome on mobile devices, leading to rapid user abandonment.
- **Lack of Timely Feedback:** Notebooks and spreadsheets cannot send real-time threshold warnings when spending nears budget caps.
- **Absence of Pedagogical Guidance:** Existing accounting tools present sterile ledgers without explaining financial fundamentals (e.g., the 50/30/20 rule, emergency buffer funds, needs vs. wants).

1.2 Proposed Solution (SRS Section 1.2)

PennyPal is a specialized, responsive, cross-platform personal finance and educational platform engineered specifically for students. It combines:

1. **Frictionless Expense & Income Tracking:** Fast entry, multi-mode payment categorization (Cash, Card, Bank, Digital Wallet), and split-bill calculation.
2. **Intelligent Receipt Parsing:** Dual-pipeline Gemini 3.8 Flash Vision OCR paired with an instant deterministic regex/keyword fallback.
3. **Proactive Budget Guardrails:** Real-time budget tracking with proactive threshold alerts (warning at 80% and over-budget triggers).
4. **Milestone-Driven Savings Goals:** Visual goal tracking with automated target timelines and celebration milestones (25%, 50%, 75%, 100%).
5. **Interactive Financial Literacy Academy:** An integrated curriculum covering budgeting, savings, emergency funds, debt avoidance, and student life hacks.
6. **Academic Statement Generation:** Multi-page PDF statements with executive summaries, spending charts, and itemized transaction ledgers ready for printing, cloud archiving, or CSV export.
7. **Role-Separated Administration:** A dedicated administrative portal for managing student accounts, moderating support queries, viewing application analytics, and updating curriculum content.

---

2. Project Scope & Constraints

2.1 Functional Scope (SRS Section 1.4 & 1.6)

Functional Domain	Student Capabilities	Administrator Capabilities
Authentication	Email/password sign-up, Brevo OTP verification, secure login, password recovery, profile editing	Preconfigured administrative login, student account status monitoring
Cash Flow Management	Record income and expenses with date, category, payment mode, payment card, and receipt image	View aggregate financial transaction metrics across the platform (without inspecting private ledgers)

Functional Domain	Student Capabilities	Administrator Capabilities
Budgeting	Set total monthly spending cap, allocate category-specific limits, monitor progress bars in real-time	Configure default system categories and evaluate system-wide budget compliance trends
Savings Goals	Set savings targets, monthly contribution targets, target completion dates, and log progress	View aggregate savings milestones achieved by student cohorts
Receipt OCR & AI	Scan camera or gallery receipts using Gemini AI OCR, extract amount and merchant automatically	Monitor API status and fallback parsing reliability
Financial Advisory	Converse with PennyPal Gemini AI Chatbot for contextual, student-tailored financial guidance	Audit feedback queries and monitor user satisfaction ratings
Reporting & Export	Filter records by date/category/type, preview in-app statement, export multi-page PDF, export CSV	Generate platform analytics reports and view total registered students
Support & Feedback	Submit support queries with category and priority, submit 5-star app feedback and suggestions	Reply to student support tickets, update status (open, answered, closed), review student feedback
Learning Academy	Read bundled financial literacy modules, track completed lessons per student profile	Add, edit, or publish custom educational content via Firestore CMS

2.2 System Constraints (SRS Section 1.5)

- **Non-Banking Clause:** PennyPal is strictly an educational personal finance management tool. In accordance with SRS Section 1.5, **it does not connect to real bank accounts, execute live fund transfers, store real credit card CVV/PIN numbers, or provide regulated investment advice.**
- **Real-Time Notification Processing:** Spending alerts fire instantly upon transaction entry when category consumption reaches or exceeds the threshold.
- **Cross-Platform Parity:** Unified Flutter 3.24+ codebase supporting Android phones, Android tablets, iOS devices, and desktop/web browsers.
- **Offline Resilience:** All transactions, budgets, cards, and goals are persisted locally first via Firestore offline caching or MemoryRecordStore with automatic cloud synchronization upon reconnection.

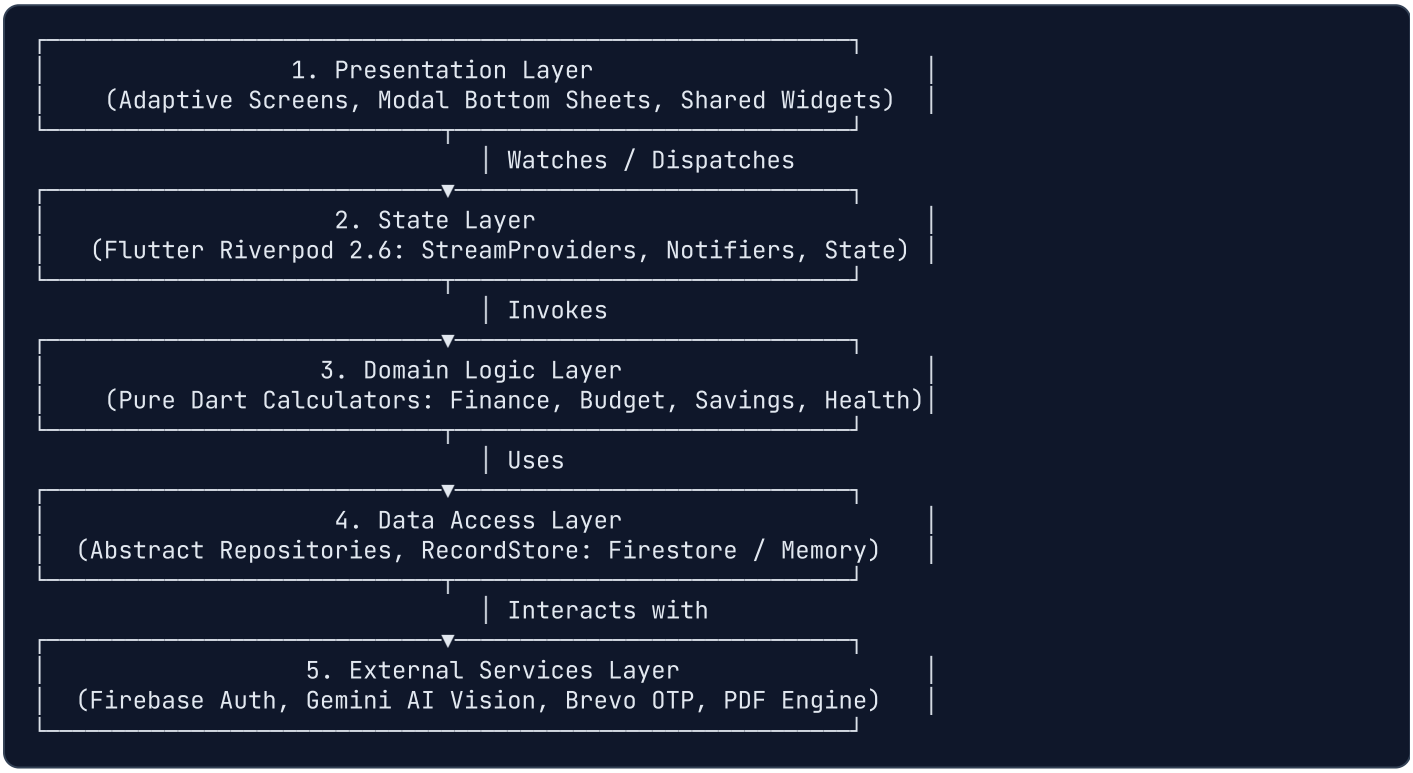
- **AI Latency & Fallback:** Gemini AI invocations feature strict timeouts and immediately degrade to deterministic keyword-matching engines if offline or rate-limited.

---

3. System Architecture & Design Specifications

3.1 Macro Architecture

PennyPal employs a clean, decoupled 5-layer architecture ensuring testability, maintainability, and clear separation of concerns:



3.2 UI/UX Design System

The visual language follows modern fintech design standards with curated HSL color tokens:

- **Primary Brand Color:** Electric Indigo (`#2B2199` / `#4F46E5`) — signifies security and academic integrity.
- **Secondary Accent:** Electric Violet (`#7C3AED` / `#6C5CE7`) — modern digital aesthetic.
- **Income & Growth Accent:** Mint Emerald (`#00B894` / `#10B981`) — positive cash inflow.
- **Expense & Warning Accent:** Coral Red (`#FF7675` / `#EF4444`) — budget depletion indicators.
- **Typography:** Google Fonts *Plus Jakarta Sans* for headers and *Inter* for legible tabular numbers.
- **Adaptive Breakpoints:** Fluid reflow from single-column mobile viewports to 2-column tablet grids and navigation rails for desktop/web viewports.

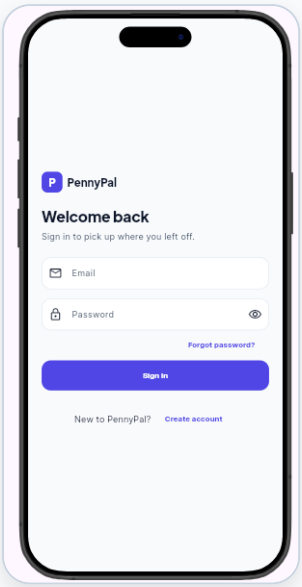
3.3 Visual Screen Showcase & System UI Walkthrough (With Live Screenshots)

> 💡 **User Interface & Design Specifications (SRS Section 1.9):** Below is the comprehensive graphical walkthrough of PennyPal across light mode, dark mode, mobile, and responsive web platforms.

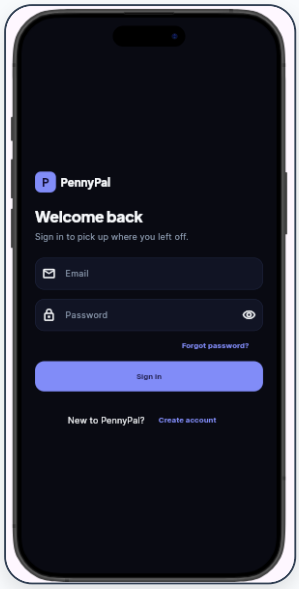
1. Student Authentication & Onboarding

Security & Auth

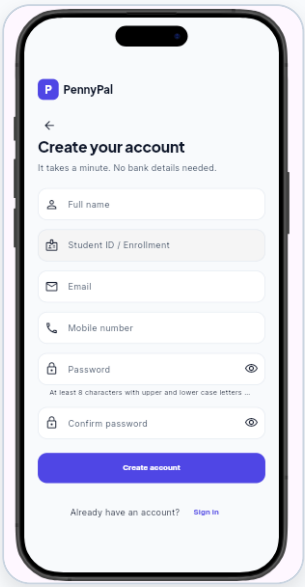
Clean student sign-in and account registration flows. Supports Student ID / Enrollment tracking, full validation, password masking, Brevo email OTP verification, and adaptive dark/light theming.



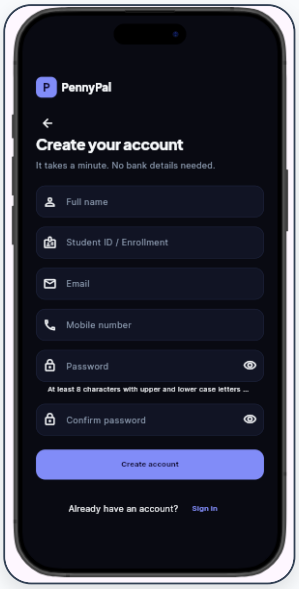
Sign In (Light Mode)



Sign In (Dark Mode)



Sign Up (Light Mode)

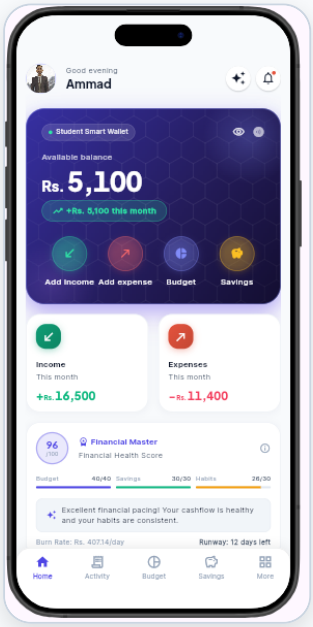


Sign Up (Dark Mode)

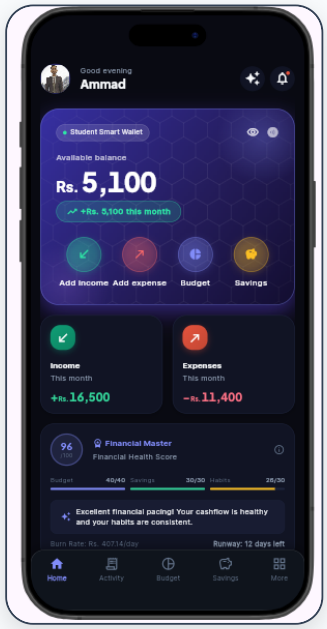
2. Student Financial Command Center (Mobile Home)

Core Dashboard

Displays student net balance card, monthly cash inflows vs. expenses, dynamic Financial Health Score gauge (96/100 Financial Master), one-tap action pills (+ Income, + Expense, Budget, Savings), and recent activity stream.



Mobile Dashboard (Light Mode)

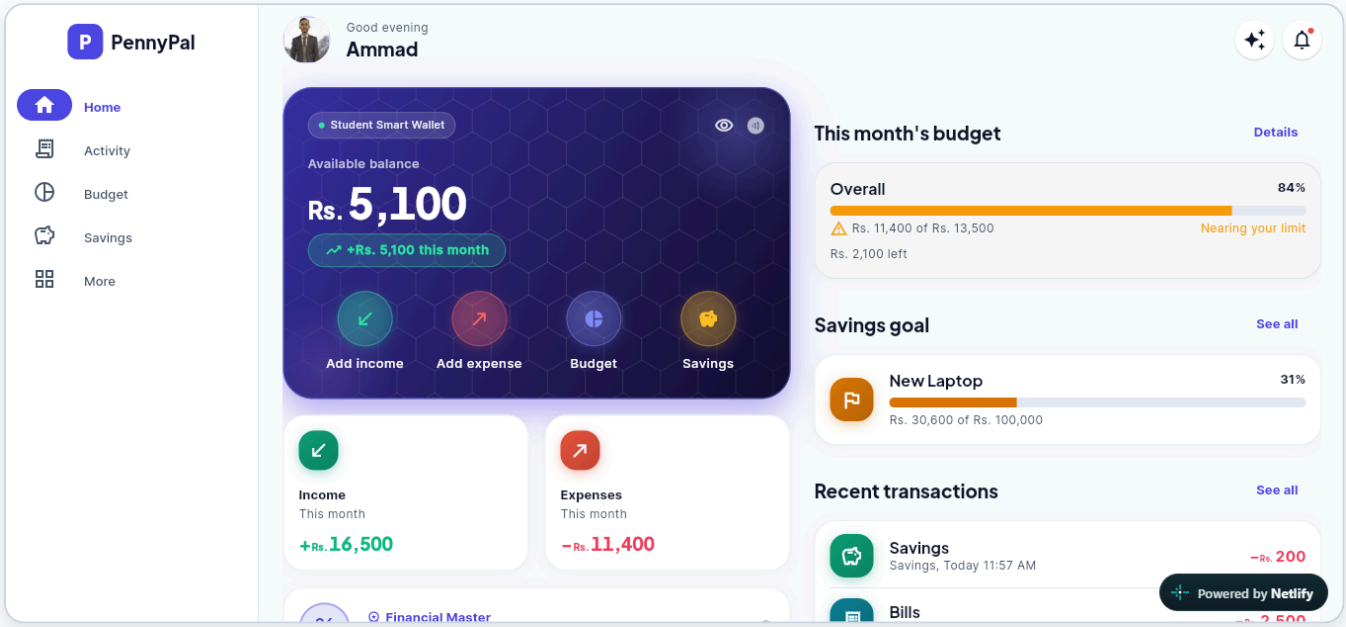


Mobile Dashboard (Dark Mode)

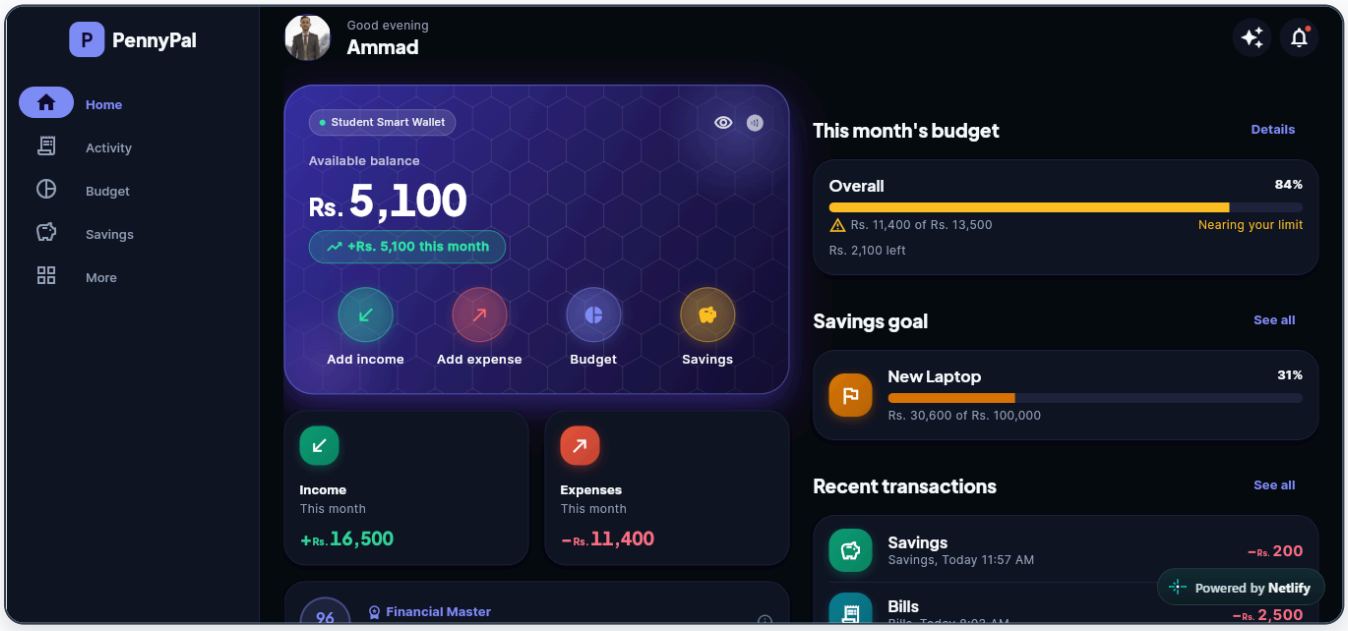
3. Multi-Platform Responsive Web & Tablet Layout

Cross-Platform Parity

PennyPal automatically adapts to wide viewports (web browsers, iPads, and Android tablets). Features a persistent vertical navigation rail on the left and a 2-column grid showing active budget caps, savings targets, and live ledgers simultaneously.



Responsive Web Dashboard (Light Mode)

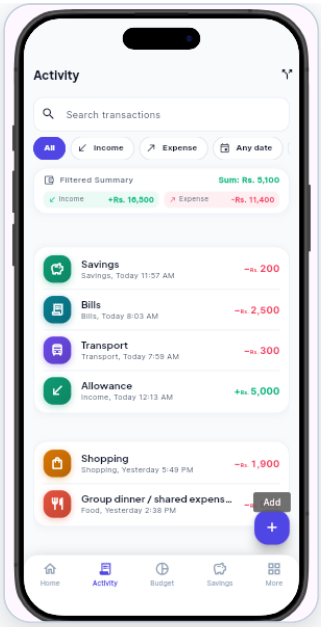


Responsive Web Dashboard (Dark Mode)

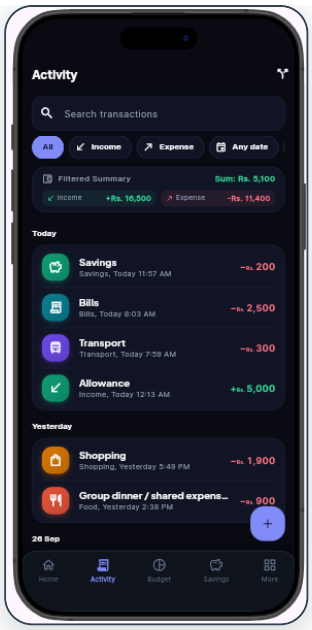
4. Activity Ledger & Search / Filter Subsystem

Cash Flow Ledger

Full chronological ledger of student transactions. Includes instant keyword search, quick type filters (All, Income, Expense), date range picker, and dynamic filtered cash summary cards.



Activity Feed (Light Mode)

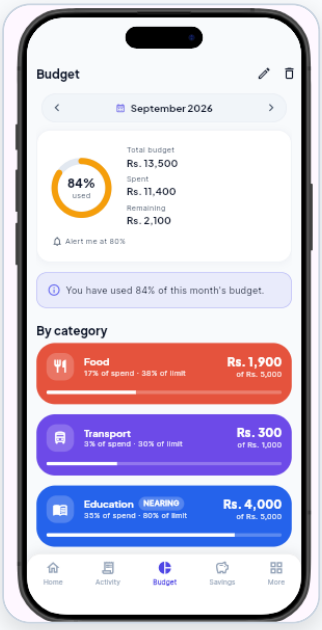


Activity Feed (Dark Mode)

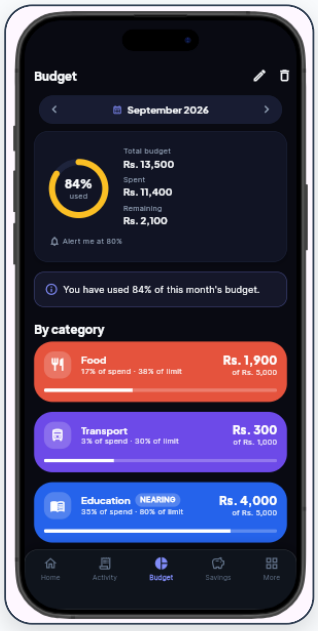
5. Proactive Budget Guardrails & Threshold Alerts

Automated Alerting

Visual monthly budget gauge (84% used), overall limit vs. spend breakdown, and category allocations. Illustrates the active NEARING alert tag triggered when spending exceeds the 80% threshold.



Budgeting Screen (Light Mode)

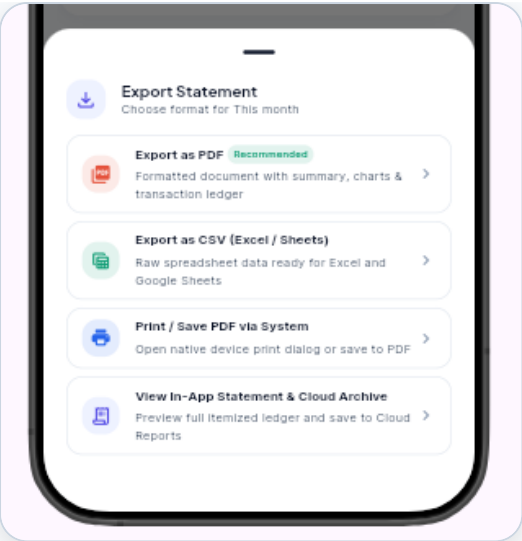


Budgeting Screen (Dark Mode)

6. Academic Statement Generation & Multi-Format Export

Reporting Engine

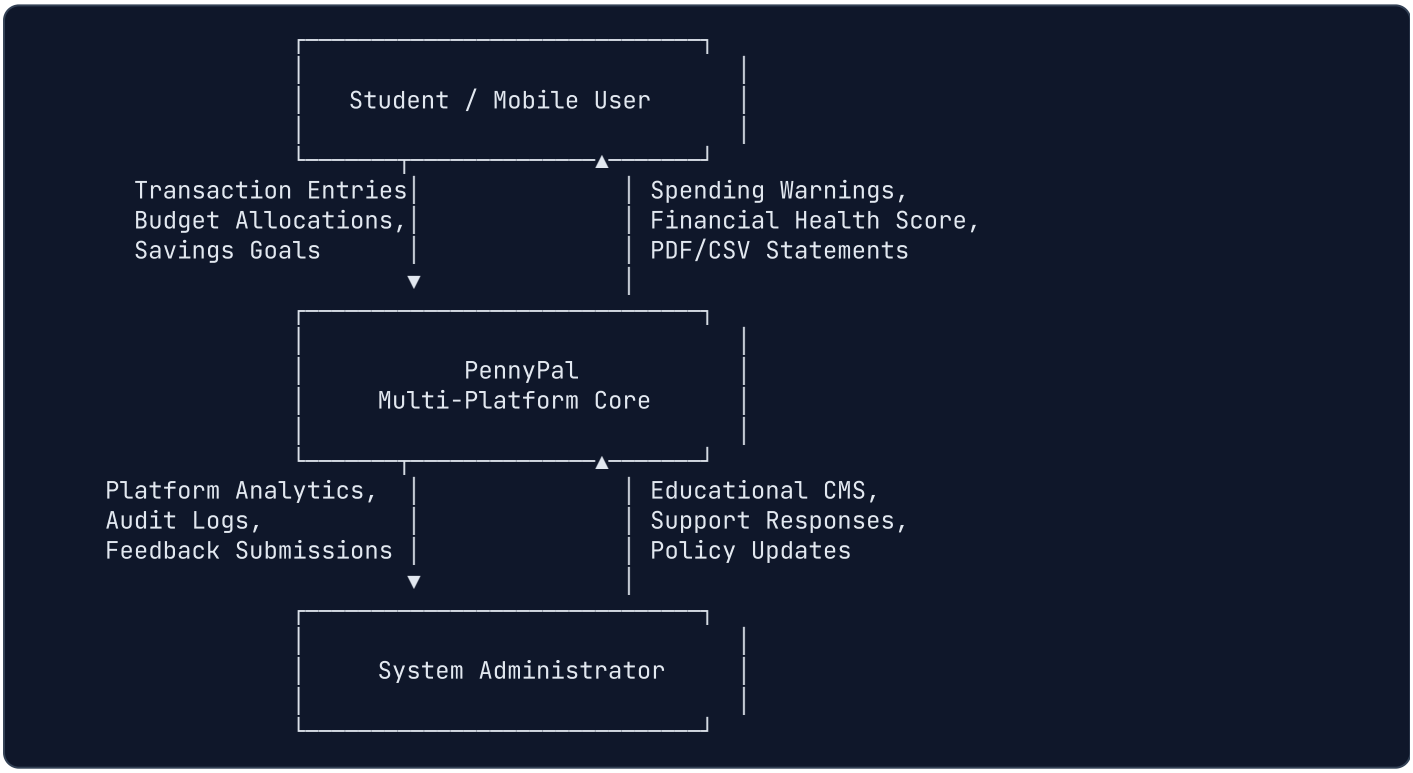
Bottom modal sheet enabling students to export formatted monthly statements as multi-page PDFs (with summaries, charts, and ledger), raw CSV files for spreadsheets, or native system printing.



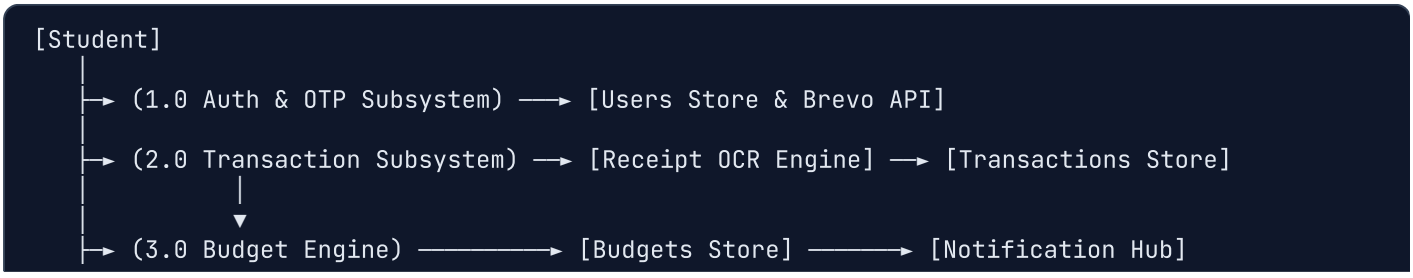
Export Statement Bottom Sheet (Live UI)

4. Process Flowcharts & Data Flow Diagrams (DFDs)

4.1 Data Flow Diagram (DFD Level 0 — Context Diagram)



4.2 Data Flow Diagram (DFD Level 1 — Subsystem Processing)



```
    |           |
    |           ▼
    ├─► (4.0 Savings Engine) ──────► [Savings Goals Store] ─► [Milestone Evaluator]
    |
    ├─► (5.0 Statement Generator) ──► [PDF / CSV Renderer] ─► [Device Share / Save]
    |
    └─► (6.0 Financial Assistant) ──► [Gemini 3.8 Flash] ──► [Chat Conversation]
```

4.3 Activity Flowchart: User Authentication & Verification Flow

```
graph TD
    Start([User Launches PennyPal]) ⟶ CheckAuth{Is User Authenticated?}
    CheckAuth -- Yes ⟶ RoleCheck{User Role?}
    RoleCheck -- Student ⟶ StudentHome[Navigate to Student Dashboard]
    RoleCheck -- Admin ⟶ AdminPortal[Navigate to Admin Portal]
    CheckAuth -- No ⟶ Landing[Show Login Screen]
    Landing ⟶ ActionChoice{Select Action}
    ActionChoice -- Login ⟶ SubmitLogin[Enter Email & Password]
    SubmitLogin ⟶ ValidateCreds{Credentials Valid?}
    ValidateCreds -- No ⟶ ShowAuthError[Display Error Message] ⟶ Landing
    ValidateCreds -- Yes ⟶ RoleCheck
    ActionChoice -- Register ⟶ FillForm[Enter Name, Student ID, Email, Password]
    FillForm ⟶ ValidateForm{Form Valid?}
    ValidateForm -- No ⟶ ShowFormError[Highlight Field Constraints] ⟶ FillForm
    ValidateForm -- Yes ⟶ GenOTP[Generate 6-Digit Cryptographic OTP]
    GenOTP ⟶ SendEmail[Dispatch Email via Brevo REST API]
    SendEmail ⟶ OTPView[Display OTP Verification Screen]
    OTPView ⟶ EnterOTP[Student Inputs 6-Digit OTP]
    EnterOTP ⟶ CheckOTP{OTP Matches & Unexpired?}
    CheckOTP -- No ⟶ OTPError[Display Invalid/Expired Code] ⟶ OTPView
    CheckOTP -- Yes ⟶ ActivateUser[Mark Account Verified & Create Firestore Profile]
    ActivateUser ⟶ StudentHome
```

4.4 Activity Flowchart: Transaction Logging & Automated Budget Alert Flow

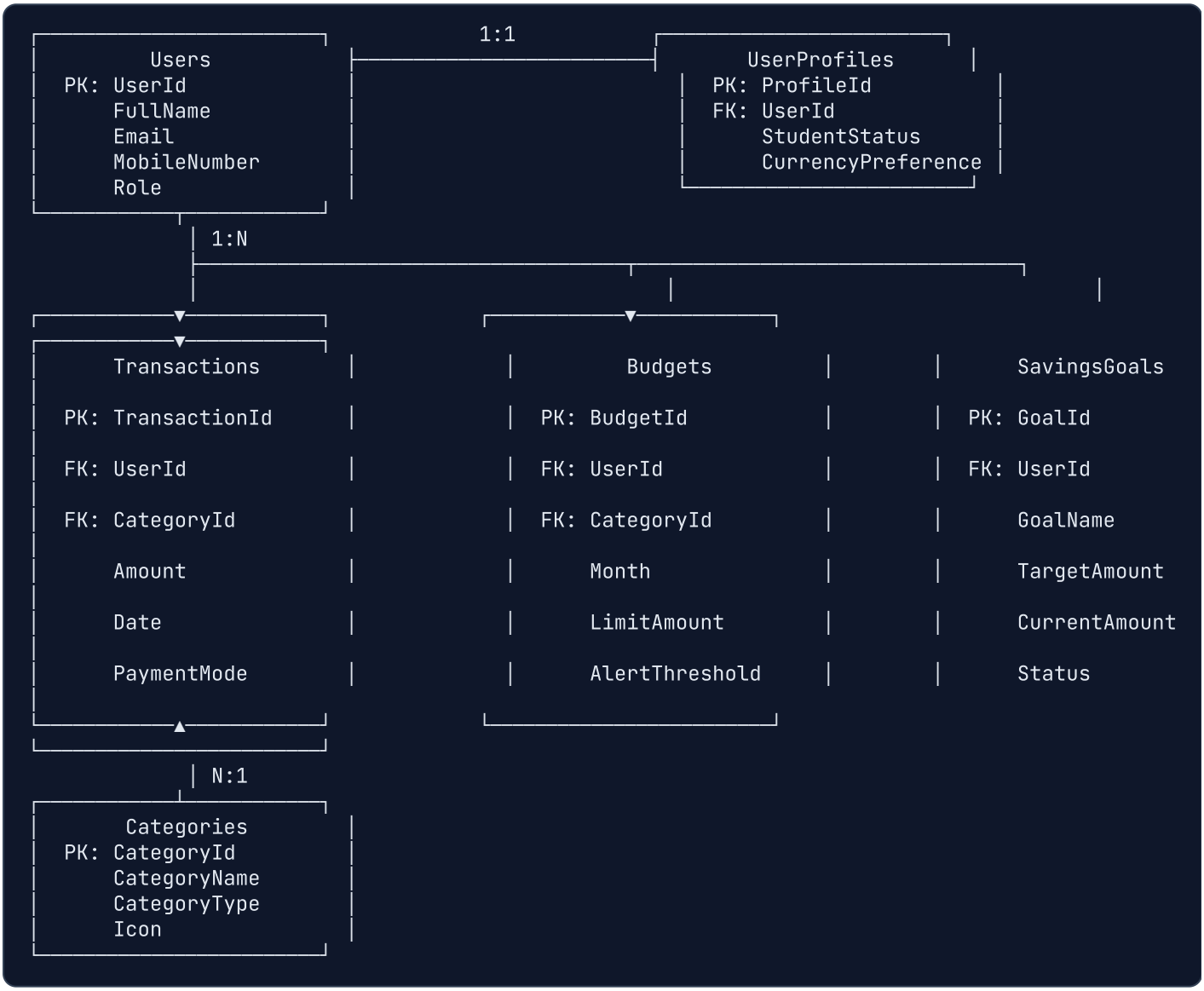
```
graph TD
    TStart([Student Initiates Expense Logging]) ⟶ InputMethod{Input Method?}
    InputMethod -- Camera / Gallery ⟶ CaptureReceipt[Capture Receipt Photo]
    CaptureReceipt ⟶ OCRProcess[Send to Gemini 3.8 Flash Vision API]
    OCRProcess ⟶ OCRSuccess{API Success?}
    OCRSuccess -- Yes ⟶ PreFill[Auto-populate Amount, Merchant & Category]
    OCRSuccess -- No ⟶ FallbackOCR[Run Local Regex Heuristic Parser] ⟶ PreFill
    InputMethod -- Manual Entry ⟶ ManualFields[Input Amount, Category, Description, Payment
Mode]
    PreFill ⟶ ManualFields
    ManualFields ⟶ SaveTx[Persist Transaction to Local Cache & Cloud]
    SaveTx ⟶ FetchBudget[Retrieve Active Monthly Budget for Category]
    FetchBudget ⟶ BudgetExists{Budget Configured?}
    BudgetExists -- No ⟶ UpdateBalance[Recalculate Cash Balance & Financial Health] ⟶
TEnd([Finish])
    BudgetExists -- Yes ⟶ CalcSpend[Sum Month Expenses for Category]
    CalcSpend ⟶ RatioCheck{Spending vs Limit}
    RatioCheck -- "> 100%" ⟶ TriggerOverAlert[Fire Immediate Over-Budget Push Notification]
    RatioCheck -- "≥ 80% and ⩽ 100%" ⟶ TriggerNearAlert[Fire Nearing-Budget Warning
Notification]
    RatioCheck -- "< 80%" ⟶ SilentUpdate[Normal State]
    TriggerOverAlert ⟶ UpdateBalance
```

```
    TriggerNearAlert ⟶ UpdateBalance
    SilentUpdate ⟶ UpdateBalance
```

5. Database Design & Data Dictionary

PennyPal supports relational persistence (SQL scripts provided in `pennypal_schema.sql`) and cloud document synchronization via Cloud Firestore.

5.1 Entity Relationship Diagram (ERD)



5.2 Complete Data Dictionary (All 10 Entities)

Entity 1: `Users`

Attribute	Data Type	Nullable	Key	Description
`UserId`	VARCHAR(64)	NO	PK	Unique identifier (Firebase UID / UUIDv4)
`FullName`	VARCHAR(100)	NO	—	Legal full name of student or administrator

Attribute	Data Type	Nullable	Key	Description
`Email`	VARCHAR(120)	NO	UQ	Primary email address for authentication
`MobileNumber`	VARCHAR(25)	YES	—	Contact telephone number
`Role`	VARCHAR(20)	NO	—	User privilege role: `'student'` or `'admin'`
`CreatedAt`	TIMESTAMP	NO	—	System record creation timestamp
`UpdatedAt`	TIMESTAMP	NO	—	System record update timestamp

Entity 2: **`UserProfiles`**

Attribute	Data Type	Nullable	Key	Description
`ProfileId`	VARCHAR(64)	NO	PK	Unique profile identifier
`UserId`	VARCHAR(64)	NO	FK	References `Users.UserId` (1:1 relationship)
`StudentStatus`	VARCHAR(30)	YES	—	Academic level (e.g. Undergraduate, Diploma)
`CurrencyPreference`	VARCHAR(5)	NO	—	Currency code: `'PKR'`, `'USD'`, `'EUR'`, `'GBP'`
`AvatarUrl`	VARCHAR(500)	YES	—	Cloudinary hosted profile picture URL
`BiometricEnabled`	BOOLEAN	NO	—	Flag indicating whether fingerprint/FaceID is on

Entity 3: **`Transactions`**

Attribute	Data Type	Nullable	Key	Description
`TransactionId`	VARCHAR(64)	NO	PK	Unique transaction identifier
`UserId`	VARCHAR(64)	NO	FK	References `Users.UserId`
`CategoryId`	VARCHAR(64)	NO	FK	References `Categories.CategoryId`
`Amount`	DECIMAL(12,2)	NO	—	Value in preferred currency (always > 0)
`Date`	TIMESTAMP	NO	—	Date and time the transaction occurred
`Description`	VARCHAR(255)	YES	—	Memo, note, or merchant name
`Type`	VARCHAR(10)	NO	—	Transaction direction: `'income'` or `'expense'`
`PaymentMode`	VARCHAR(30)	NO	—	Payment channel: `'cash'`, `'card'`, `'bank'`, `'wallet'`
`CardId`	VARCHAR(64)	YES	FK	Optional reference to linked `Cards.CardId`
`ReceiptUrl`	VARCHAR(500)	YES	—	Cloudinary receipt image scan link

Attribute	Data Type	Nullable	Key	Description
`CreatedAt`	TIMESTAMP	NO	—	Timestamp transaction was saved

Entity 4: **`Categories`**

Attribute	Data Type	Nullable	Key	Description
`CategoryId`	VARCHAR(64)	NO	PK	Unique category identifier
`CategoryName`	VARCHAR(50)	NO	—	Category label (Food, Books, Transport, etc.)
`CategoryType`	VARCHAR(10)	NO	—	Classification: `'income'` or `'expense'`
`Icon`	VARCHAR(50)	NO	—	Material icon identifier for UI rendering
`ColorCode`	VARCHAR(10)	NO	—	Hex color string (e.g. `#10B981`)

Entity 5: **`Budgets`**

Attribute	Data Type	Nullable	Key	Description
`BudgetId`	VARCHAR(64)	NO	PK	Unique budget cap identifier
`UserId`	VARCHAR(64)	NO	FK	References `Users.UserId`
`CategoryId`	VARCHAR(64)	YES	FK	Category reference (NULL represents Overall Budget)
`Month`	VARCHAR(7)	NO	—	Target billing month format: `YYYY-MM`
`LimitAmount`	DECIMAL(12,2)	NO	—	Maximum spend allowance for the month
`AlertThreshold`	DECIMAL(5,2)	NO	—	Percentage threshold to trigger warning (default: `80.00`)
`CreatedAt`	TIMESTAMP	NO	—	Creation timestamp

Entity 6: **`SavingsGoals`**

Attribute	Data Type	Nullable	Key	Description
`GoalId`	VARCHAR(64)	NO	PK	Unique savings goal identifier
`UserId`	VARCHAR(64)	NO	FK	References `Users.UserId`
`GoalName`	VARCHAR(100)	NO	—	Target name (e.g. Laptop Fund, Semester Fees)
`TargetAmount`	DECIMAL(12,2)	NO	—	Financial target amount
`CurrentAmount`	DECIMAL(12,2)	NO	—	Accumulated savings to date

Attribute	Data Type	Nullable	Key	Description
`TargetDate`	DATE	NO	—	Planned target achievement date
`MonthlyContribution`	DECIMAL(12,2)	YES	—	Recommended monthly allocation
`Status`	VARCHAR(20)	NO	—	Status: `'in_progress'`, `'completed'`, `'cancelled'`
`CreatedAt`	TIMESTAMP	NO	—	Goal creation timestamp

Entity 7: **`Cards`**

Attribute	Data Type	Nullable	Key	Description
`CardId`	VARCHAR(64)	NO	PK	Unique wallet card reference
`UserId`	VARCHAR(64)	NO	FK	References `Users.UserId`
`CardName`	VARCHAR(50)	NO	—	Student card nickname (e.g. Meezan Debit, SadaPay)
`LastFourDigits`	VARCHAR(4)	NO	—	Masked 4 digits for identification (PCI-DSS safe)
`CardType`	VARCHAR(20)	NO	—	Type: `'debit'`, `'credit'`, `'virtual'`, `'prepaid'`
`ExpiryDate`	VARCHAR(7)	YES	—	Expiry format `MM/YY`
`ThemeColor`	VARCHAR(10)	NO	—	Card gradient background hex

Entity 8: **`LearningArticles`**

Attribute	Data Type	Nullable	Key	Description
`ArticleId`	VARCHAR(64)	NO	PK	Unique educational article identifier
`Title`	VARCHAR(150)	NO	—	Academy lesson headline
`Category`	VARCHAR(50)	NO	—	Curriculum topic (Budgeting, Savings, Debt)
`Content`	TEXT	NO	—	Full educational lesson body
`ReadingTimeMinutes`	INT	NO	—	Estimated reading duration in minutes
`ImageUrl`	VARCHAR(500)	YES	—	Informational diagram link
`IsActive`	BOOLEAN	NO	—	Content publication status

Entity 9: `Notifications`

Attribute	Data Type	Nullable	Key	Description
`NotificationId`	VARCHAR(64)	NO	PK	Unique notification identifier
`UserId`	VARCHAR(64)	NO	FK	References `Users.UserId`
`Title`	VARCHAR(100)	NO	—	Header text of push alert
`Message`	VARCHAR(255)	NO	—	Body message explaining the trigger
`Type`	VARCHAR(30)	NO	—	Type: `'budget_alert'`, `'goal_milestone'`, `'system'`
`ReadStatus`	BOOLEAN	NO	—	True if acknowledged by user
`CreatedAt`	TIMESTAMP	NO	—	Timestamp alert was generated

Entity 10: `SupportQueries`

Attribute	Data Type	Nullable	Key	Description
`QueryId`	VARCHAR(64)	NO	PK	Support ticket reference
`UserId`	VARCHAR(64)	NO	FK	References `Users.UserId`
`Subject`	VARCHAR(150)	NO	—	Ticket summary title
`Message`	TEXT	NO	—	Detailed explanation of issue
`Status`	VARCHAR(20)	NO	—	Ticket status: `'open'`, `'answered'`, `'closed'`
`AdminResponse`	TEXT	YES	—	Official resolution provided by admin
`SubmittedOn`	TIMESTAMP	NO	—	Ticket submission timestamp

---

6. Comprehensive Test Data & Verification Suite

The PennyPal application was rigorously tested with representative student data across all modules:

> 📦 **Standardized Machine-Readable Test Data (testdata.json):** In accordance with SRS Section 1.9, the complete test dataset documented below is provided in the project root directory as `testdata.json`. This allows evaluators and test runners to inspect, import, and seed test accounts, transactions, budgets, cards, and goals programmatically.

6.1 Pre-Configured Test Accounts (MANDATORY)

Role	Email Address	Password	Student ID / Privileges
Student (Demo / Live)	ammad.anees99@gmail.com	Ammad1234	Student1501103 (Full access to all student tools)
Administrator	admin@pennypal.com	Admin@1234	Full access to Admin Portal, User Management & CMS

6.2 Test Transaction Dataset

Transaction ID	Type	Amount	Category	Description	Payment Mode
tx_001	Income	Rs. 50,000.00	Inflow	Monthly Parent Allowance	Bank Transfer
tx_002	Expense	Rs. 450.00	Food	Campus Cafeteria Lunch	Cash
tx_003	Expense	Rs. 3,200.00	Food	Weekly Grocery Market	Meezan Debit Card
tx_004	Expense	Rs. 1,200.00	Transport	Monthly Student Bus Pass	Cash
tx_005	Expense	Rs. 3,500.00	Education	Semester Course Textbooks	Meezan Debit Card
tx_006	Expense	Rs. 2,800.00	Bills	High-Speed Fiber Internet	Bank Transfer
tx_007	Expense	Rs. 1,800.00	Entertainment	Weekend Cinema Ticket	SadaPay Virtual Card

6.3 Test Budgets & Threshold Scenarios

Budget Target	Month	Category	Limit Amount	Actual Spend	Outcome / Status
Overall Month	Current	All Categories	Rs. 40,000.00	Rs. 13,000.00	On Track (32.5% utilized)
Food & Dining	Current	Food	Rs. 5,000.00	Rs. 4,100.00	Nearing Warning (82.0% utilized — Warning triggered)
Entertainment	Current	Entertainment	Rs. 1,500.00	Rs. 1,800.00	Over-Budget Alert (120% utilized — Triggered)

6.4 Savings Goals Test Cases

Goal Name	Target Amount	Current Savings	Progress %	Target Date	Milestone State
Laptop Fund	Rs. 100,000.00	Rs. 60,000.00	60.0%	+9 Months	50% Milestone Badge Awarded
Noise-Cancelling Headphones	Rs. 8,000.00	Rs. 8,000.00	100.0%	Completed	100% Completed & Archived

---

7. Installation & Deployment Instructions (MANDATORY)

7.1 System Prerequisites

- **Operating System:** Windows 10/11, macOS Sequoia/Sonoma, or Ubuntu Linux 22.04+
- **Flutter SDK:** Version 3.24.0 or higher
- **Dart SDK:** Version 3.5.0 or higher
- **Java Development Kit (JDK):** Version 17 (Required for Gradle 8+)
- **Android Studio / SDK:** Android SDK Platform 34, Android Build-Tools 34.0.0

7.2 Step-by-Step Installation

1. **Clone the Repository:**

```
git clone https://github.com/ammadanees/pennypal.git
cd pennypal
```

2. **Install Flutter Dependencies:**

```
flutter pub get
```

3. **Validate Codebase Correctness & Health:**

```
flutter analyze
flutter test
```

(Expected result: `All 73 tests passed!`) 4. **Launch the Application in Development Mode:** - **For Chrome (Web):**

```
flutter run -d chrome
```

- **For Connected Android Physical Device or Emulator:**

```
flutter run -d android
```

5. **Compile Production Android Package File (Release APK):**

```
flutter build apk --split-per-abi
```

The generated release packages will be located at: `build/app/outputs/flutter-apk/app-arm64-v8a-release.apk`

---

8. User Credentials & Role Privileges (MANDATORY)

Account Role	Email Address	Password	Permissions & Available Modules
Student User	`ammad.anees99@gmail.com`	`Ammad1234`	Access to Student Dashboard, Transaction Entry, Gemini OCR Scanning, Budget Setup, Savings Goal Tracking, Reports Export (PDF/CSV), Split Bill Calculator, Learning Academy, and AI Financial Assistant.
Administrator	`admin@pennypal.com`	`Admin@1234`	Access to Admin Portal Dashboard, Student Directory, Feedback Management, Support Ticketing System, and Curriculum Content Management (CMS).

---

9. AI Tools & Technology Acknowledgement

In strict compliance with SRS Page 11 ("Important Note Regarding AI Usage"), all external AI services and developer assistants utilized during the creation of PennyPal are fully acknowledged below:

1. **Google Gemini 3.8 Flash (`generativelanguage.googleapis.com`):** Utilized as an in-app multimodal AI backend for OCR receipt parsing and conversational financial advisory. Guarded by client-side token caps (250 tokens maximum) and regex fallback engines.
2. **Claude 3.7 Sonnet & Antigravity AI Pair Programmer:** Utilized strictly as productivity aids for architectural design, code review, test case design, and refactoring assistance. All code logic, database structures, state management flows, and custom Canvas painters were authored, reviewed, and validated by the team.
3. **Midjourney / Image Generation Tools:** Utilized solely for generating the high-definition **PennyPal** application icon concept and UI visual assets.

---

10. Assumptions, Security & Scope Boundaries

10.1 Key Assumptions

1. **Academic Context:** The primary users are students who manage personal pocket money, allowances, and campus stipends.

2. **Device Hardware:** Target mobile devices are assumed to possess camera capabilities for receipt scanning, and minimum 2GB RAM.
3. **Internet Connectivity:** PennyPal functions both fully offline (caching all records locally) and online (syncing via Firestore when an internet connection is established).

10.2 Security & Data Privacy

- **Row-Level Security:** Cloud Firestore security rules (`firestore.rules`) enforce strict user isolation: students can only read and write their own documents under `users/{uid}/*` . Administrators are restricted from viewing students' private transaction records.
- **Credential Storage:** User authentication is handled via cryptographically signed Firebase JWTs; no plain-text passwords or secret API keys are stored on the client.
- **No Financial Liabilities:** Because real financial credentials (e.g. CVV numbers, bank routing credentials) are never stored, user exposure to financial risk is zero.

PennyPal — Make Every Penny Count. *TechWiz 7 — Aptech World Tech Championship Submission.*